

CSE-2421 - Computer Algorithm

Final Syllabus

Segment 4: Dynamic programming (advanced)

Min Cost Path in a Grid: Moving from top-left to bottom-right with minimum cost; Longest Increasing Subsequence (LIS): Find the longest subsequence that is strictly increasing; Longest Common Subsequence (LCS): Find the longest subsequence common to two sequences; Matrix Chain Multiplication: Find the most efficient way to multiply a chain of matrices.

Segment 5: M-way trees and B-trees, Amortized Analysis

M-way & B-trees: M-way search tree structure and properties; B-tree definition and invariants; Insertion and node splitting; Deletion operations including borrowing and merging; Height analysis and Disk I/O optimization; Applications in Database Indexing (MySQL) and File Systems (NTFS).

Amortized Complexity: Concept of amortized vs. worst-case analysis; Aggregate method of analysis; Accounting (Banker's) method and potential functions; Dynamic Array implementation; Analysis of STL vector growth strategies (doubling vs. 1.5x); Cost-benefit trade-offs in memory reallocation.

Segment 6: Graphs, basic traversal techniques and MST

Graphs and basic traversal techniques: representation of graphs; breadth-first search; depth-first search; application of BFS and DFS: shortest paths, component count, topological sort.

Minimum spanning tree: disjoint set data structure; Kruskal's algorithm; Prim's algorithm.

Segment 7: Shortest-path algorithms

Single-source shortest-path algorithms: Dijkstra's algorithm; Bellman-Ford's algorithm; All-pairs shortest-path algorithms: Warshall's algorithm for transitive closure; Floyd's algorithm.

Segment 8: Max-Flow, NP-Completeness, Backtracking, Branch and Bound

Maximum flow: Ford-Fulkerson method, Edmonds-Karp algorithm; Theory of NP-Completeness, P, NP, NP-Complete and NP-Hard Problems; Backtracking: Permutations, N-Queen Problem; Branch and Bound: TSP

Practice Questions

Segment 4: Dynamic programming (advanced)

Topic 1: Min Cost Path in a Grid

No.	Question																
1.	Consider an $m \times n$ grid in which every cell (i, j) holds a non-negative cost. A robot starts at the top-left cell and may move only one step right or one step down until it reaches the bottom-right cell. The cost of a path is the sum of the costs of all cells visited. (a) Define the dynamic programming state for the minimum-cost path. (b) Write the recurrence relation. (c) State the base cases. (d) State the time and space complexity of the bottom-up solution.																
2.	For the cost grid below, movement is allowed only right or down, from the top-left cell to the bottom-right cell. <table><tr><td>1</td><td>3</td><td>1</td><td>2</td></tr><tr><td>1</td><td>5</td><td>1</td><td>3</td></tr><tr><td>4</td><td>2</td><td>1</td><td>1</td></tr><tr><td>1</td><td>2</td><td>1</td><td>1</td></tr></table> (a) Using bottom-up dynamic programming, compute the minimum path cost. Show the filled DP table as it is built. (b) List the cells visited on one optimal (minimum-cost) path. (c) If diagonal moves (one step down-and-right) were also permitted, rewrite the recurrence relation.	1	3	1	2	1	5	1	3	4	2	1	1	1	2	1	1
1	3	1	2														
1	5	1	3														
4	2	1	1														
1	2	1	1														
3.	A student proposes a greedy method for the grid minimum-cost path problem: at each cell, move to whichever neighbour (right or down) has the smaller cost. (a) Construct a small grid (give the cell values) on which this greedy method does NOT produce the minimum-cost path. (b) For your grid, show both the cost of the greedy path and the true minimum cost. (c) Briefly explain why the greedy choice fails here while the dynamic programming approach succeeds.																

Topic 2: Longest Increasing Subsequence (LIS)

No.	Question
4.	Given an array of n integers, the Longest Increasing Subsequence (LIS) is the longest subsequence whose elements appear in strictly increasing order. (a) Explain why solving LIS by examining all possible subsequences leads to exponential running time. (b) Define the DP state and write the $O(n^2)$ recurrence relation for the length of the LIS. (c) Briefly describe the key idea that allows LIS to be solved in $O(n \log n)$ time.
5.	Consider the sequence $A = [3, 10, 2, 1, 20, 4, 6, 8]$. (a) Compute the array $L[i]$ = length of the longest increasing subsequence ending at index i. Show every value. (b) State the length of the overall LIS. (c) Reconstruct and write one longest increasing subsequence of A.
6.	You are given n envelopes, each described by a (width, height) pair. Envelope B fits inside envelope A only if the width and height of B are both strictly smaller than those of A. The goal is to find the maximum number of envelopes that can be nested one inside another to form a chain. (a) Describe how this problem reduces to the Longest Increasing Subsequence problem. (b) State the preprocessing (sorting) step required before LIS can be applied. (c) Define the DP state and the recurrence used after preprocessing.

Topic 3: Longest Common Subsequence (LCS)

No.	Question
7.	This question concerns the Longest Common Subsequence (LCS) of two strings $X[1..m]$ and $Y[1..n]$. (a) Distinguish between a subsequence and a substring, using a short example. (b) Write the recurrence for the LCS length, clearly stating all three cases (empty string, matching characters, mismatching characters). (c) Explain how the LCS problem exhibits both optimal substructure and overlapping subproblems.
8.	Let $X = \text{"ABCBBDAB"}$ and $Y = \text{"BDCAB"}$. (a) Fill the LCS dynamic programming table for X and Y. (b) State the length of the LCS.

No.	Question
	(c) Reconstruct one LCS by tracing back through the table. (d) Is the LCS unique? If not, give a second valid LCS of the same length.

Topic 4: Matrix Chain Multiplication

No.	Question
9.	This question concerns the Matrix Chain Multiplication problem. (a) State the Matrix Chain Multiplication problem. (b) Using a small example, explain why the order of multiplication changes the number of scalar multiplications even though the final product matrix is the same. (c) Write the DP recurrence $m[i][j]$ for the minimum number of scalar multiplications and identify the optimal substructure it relies on.
10.	Four matrices are to be multiplied with the following dimensions: $A_1 : 5 \times 4$, $A_2 : 4 \times 6$, $A_3 : 6 \times 2$, $A_4 : 2 \times 7$ (dimension array $p = [5, 4, 6, 2, 7]$). (a) Using bottom-up dynamic programming, compute the $m[i][j]$ table for the minimum number of scalar multiplications. (b) State the minimum number of scalar multiplications required. (c) Give the optimal parenthesization of the chain.
11.	Consider fully parenthesizing the product of a chain of n matrices. (a) Explain why the number of distinct full parenthesizations grows extremely fast with n (you may refer to the Catalan number), and why a brute-force search over all parenthesizations is impractical. (b) State the time and space complexity of the dynamic programming solution to Matrix Chain Multiplication. (c) Justify, in terms of overlapping subproblems, why dynamic programming is far more efficient than the brute-force approach.

Segment 5: M-way trees and B-trees, Amortized Analysis

M-way trees and B-trees

1. Explain why an M-way search tree can reduce the height of a tree compared with a binary search tree, and analyze how storing multiple keys in one node affects the number of comparisons during searching.
2. In an M-way search tree of order 5, describe how keys and child pointers are arranged inside a node, and explain why a node with 4 keys must have 5 possible child directions.
3. Construct an M-way search tree of order 4 using the keys 50, 20, 70, 10, 30, 60, 80, 25, 35, 65, 90, and explain whether the final tree is necessarily balanced or not.
4. Compare a normal M-way search tree and a B-tree in terms of balance, minimum key requirement, node utilization, and suitability for disk-based storage.
5. Explain why B-trees are considered self-balancing M-way search trees, and analyze how this property helps in maintaining efficient search, insertion, and deletion operations.
6. For a B-tree of minimum degree ($m=3$), determine the minimum and maximum number of keys and children allowed in each non-root node, and explain why these limits are important for maintaining balance.
7. Insert the keys 10, 20, 5, 6, 12, 30, 7, 17, 3, 25, 40 into a B-tree of minimum degree ($t=2$), and identify at which insertions node splitting occurs.
8. Explain what happens when a B-tree node becomes full during insertion, and analyze why the middle key is promoted instead of promoting the smallest or largest key..
9. Insert the keys 40, 20, 60, 10, 30, 50, 70, 5, 15, 25, 35, 45 into a B-tree of minimum degree ($t=3$), and show the final tree structure.
10. Consider a B-tree of minimum degree $t=3$. Insert the following keys into the B-tree in the given order:

10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120, 130, 140, 150

After constructing the B-tree, perform the following deletions one by one: 30, 60, 90

Show the B-tree after each deletion and identify which deletion case is used in each step:

- ✓ deletion from a leaf node without underflow,
 - ✓ deletion using borrowing from a sibling,
 - ✓ deletion using merging with a sibling.
11. Explain the difference between deleting a key from a leaf node and deleting a key from an internal node in a B-tree, and discuss why internal node deletion is more complex.
 12. A B-tree deletion causes a node to have fewer than the minimum number of keys; analyze when borrowing from a sibling is possible and when merging becomes necessary.
 13. Analyze why B-trees are more efficient than binary search trees for database indexing, especially when the data is stored on disk rather than in main memory.
 14. A database table contains 1,000,000 records and uses a B-tree index; explain how high branching factor reduces disk I/O and why searching does not require scanning all records.

15. Explain how MySQL can use a B-tree index to speed up searching, sorting, and range queries, and analyze why this is useful for primary key indexing.
16. A student says, “A B-tree is just a binary search tree with more keys per node.” Do you agree with this statement? Justify your answer using balance, node capacity, and disk access behavior.
17. Analyze the trade-off between using a large order in a B-tree and using a small order, considering tree height, number of comparisons inside a node, memory usage, and disk block efficiency.

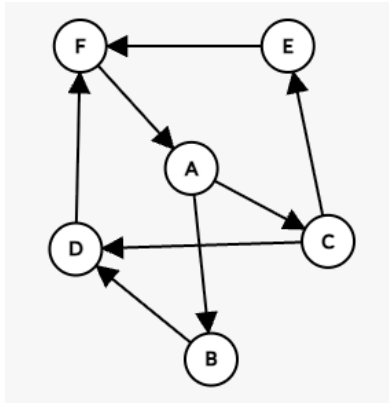
Amortized Analysis

18. Explain the difference between **worst-case complexity** and **amortised complexity** using dynamic array insertion as an example.
19. Why is insertion at the end of a dynamic array considered **amortised ($O(1)$)**, although resizing may take ($O(n)$) time?
20. A dynamic array has size 8 and capacity 8. If one more element is inserted, explain what happens during reallocation.
21. Explain the **aggregate method** of amortised analysis in simple words.
22. What is the main idea of the **accounting method** or **Banker's method** in amortised analysis?
23. Why do we charge extra cost for some cheap operations in the accounting method?
24. Explain the idea of a **potential function** in amortised analysis using dynamic array resizing.
25. Compare **doubling growth** and **1.5x growth** in a vector in terms of memory wastage.
26. Compare **doubling growth** and **1.5x growth** in a vector in terms of reallocation frequency.
27. A student says, “Vector insertion is always ($O(n)$) because resizing copies all elements.” Do you agree? Justify your answer using amortised analysis.

Segment 6: Graphs, basic traversal techniques and MST

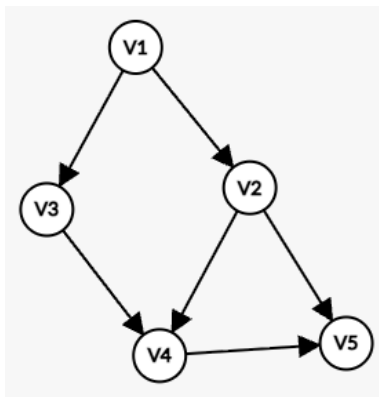
1. Compare the Adjacency Matrix and Adjacency List representations of a graph $G = (V, E)$ in terms of:
 - Memory storage requirements for sparse vs. dense graphs.
 - Time complexity to check if a specific edge (u, v) exists.
 - Time complexity to find all neighbors of a given vertex u .
2. Explain why Breadth-First Search (BFS) is guaranteed to find the shortest path (in terms of the minimum number of edges) from a source vertex to all other vertices in an unweighted graph. Why does this property fail if edges have varying positive weights?
3. Describe an efficient algorithm based on DFS or BFS to find the total number of connected components in an undirected graph $G = (V, E)$. Provide the pseudocode and state its overall time complexity in terms of V and E .

4. Refer to the graph described below:



- **(a)** List the order in which vertices are visited during a Breadth-First Search (BFS) starting from vertex A. Assume neighbors are visited in alphabetical order.
- **(b)** List the order in which vertices are visited during a Depth-First Search (DFS) starting from vertex A.

5. Refer to the graph described below:



- **(a)** Provide two valid topological orderings for this graph.
- **(b)** Explain how Kahn's algorithm (indegree-based approach) detects if a given directed graph contains a cycle.

6. Explain how the concept of graph 2-coloring can be used to check if a graph is bipartite. How does a DFS traversal immediately identify the exact edge that violates the bipartite property if the graph is not bipartite?

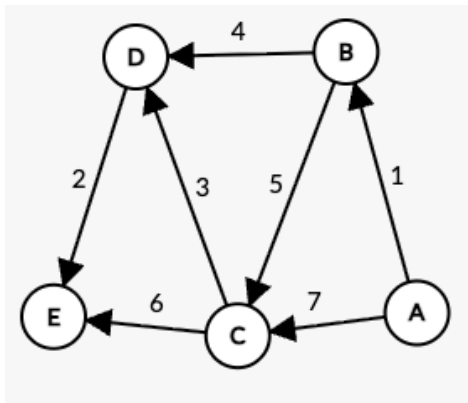
7. Not all Directed Acyclic Graphs (DAGs) have a unique topological ordering. Describe how you can modify Kahn's algorithm (the indegree-based approach) to detect whether a given DAG has a unique topological sort during execution.

8. The diameter of an unweighted tree is the longest path (in terms of number of edges) between any pair of vertices. Give an example of a graph with diameter 6 and explain the algorithm of how to calculate this diameter from a given graph.

9. **(a)** Define what a Minimum Spanning Tree is.

(b) When will a graph contain unique MST? Give example and counter example of when a graph contains unique MST and when contains multiple possible MSTs.

10. Execute Prim's algorithm (starting at vertex A) and Kruskal's algorithm on the graph described below:



- (a) List the order with appropriate steps in which edges are added to the tree using Kruskal's algorithm.
- (b) List the order with appropriate steps in which edges are added to the tree using Prim's algorithm.

11. Let G be a connected, weighted graph. Suppose we modify the graph by applying a function $f(w)$ to every edge weight w . State with a brief justification whether the Minimum Spanning Tree structure changes if:

- (a) $f(w) = w + c$ (where c is a positive constant).
- (b) $f(w) = w^2$ (assuming all original weights are positive).

12. Describe how you would modify Kruskal's algorithm to find a **Maximum** Spanning Tree instead of a Minimum Spanning Tree.

13. Let T be a Minimum Spanning Tree of a connected, weighted graph G . Suppose we increase the weight of a single edge e that belongs to T by a positive value k . Describe a systematic way to determine whether T remains a Minimum Spanning Tree of the modified graph.

14. Consider a connected, undirected graph G with positive edge weights. If we square the weight of every edge, will the Minimum Spanning Tree (MST) of the graph change? Justify your answer.

15. What is the time complexity of BFS and DFS on a graph with V vertices and E edges?

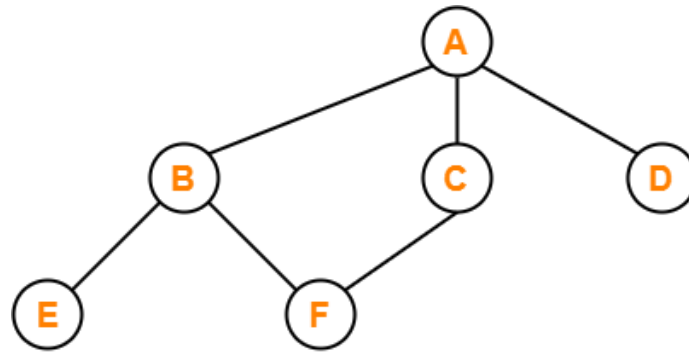
Can DFS be used to detect a cycle in a directed graph? How?

16. Why is BFS preferred over DFS for finding the shortest path in an unweighted graph?

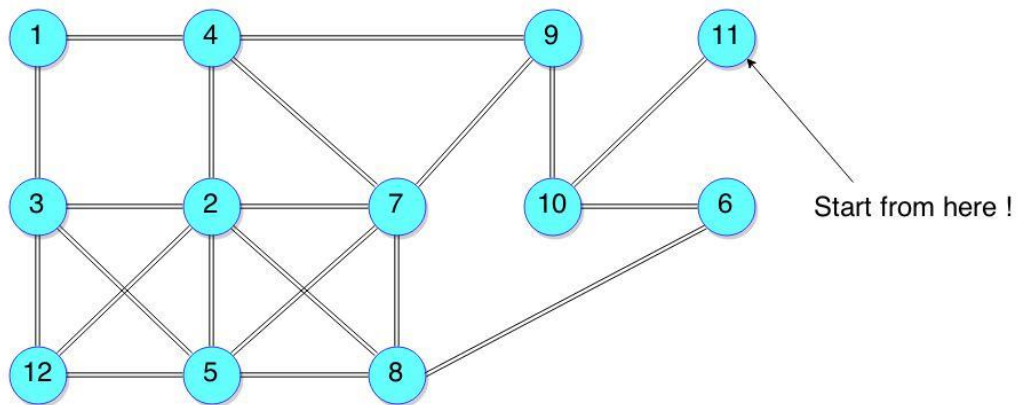
How can BFS be used to count connected components in an undirected graph?

17. Can BFS and DFS be applied on weighted graphs? Why or why not?

18. Apply BFS and DFS on the graph below. Source is "E".



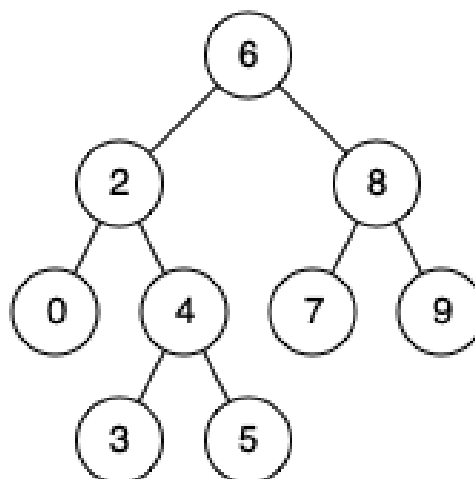
19. Apply BFS and DFS on the graph below.



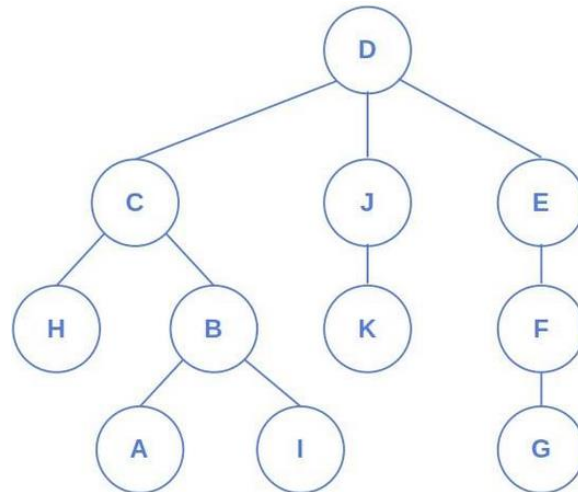
20. What is a Minimum Spanning Tree (MST)? How many edges does it contain for a graph with V vertices?

21. What is the key difference between Kruskal's and Prim's algorithms in terms of their approach? Which algorithm is better for dense graphs: Kruskal's or Prim's? Why?

22. Find out $LCA(0,5)$; $LCA(4,9)$ using both brute-force and recursive technique.



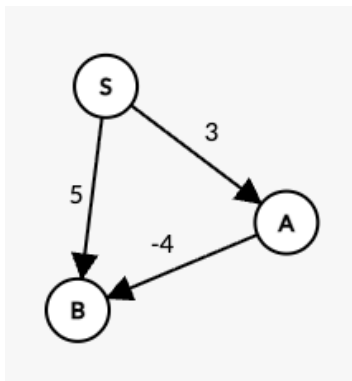
23. What is the diameter of a tree? Find the diameter of given tree with path.



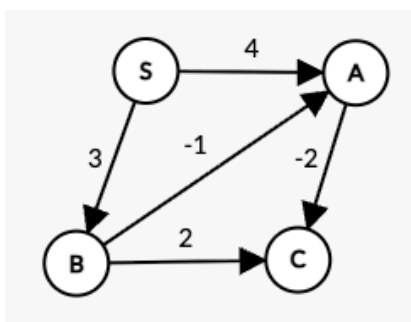
Segment 7: Shortest-path algorithms

1. Explain why Dijkstra's algorithm can fail to produce correct shortest paths when a graph contains negative-weight edges, even if there are no negative-weight cycles.

- Trace Dijkstra's algorithm on the following graph to show where it fails:



2. Trace the Bellman-Ford algorithm on the following directed graph with vertex S as the source:



- Show the values of the distance array after each of the required iterations.

3. You are given a directed graph and a subset of vertices designated as "start nodes." You need to find the shortest path from any of these start nodes to a single destination vertex d . Explain how you can transform this multiple-source shortest path problem into a standard single-source shortest path problem so that it can be solved with a single execution of Dijkstra's algorithm.

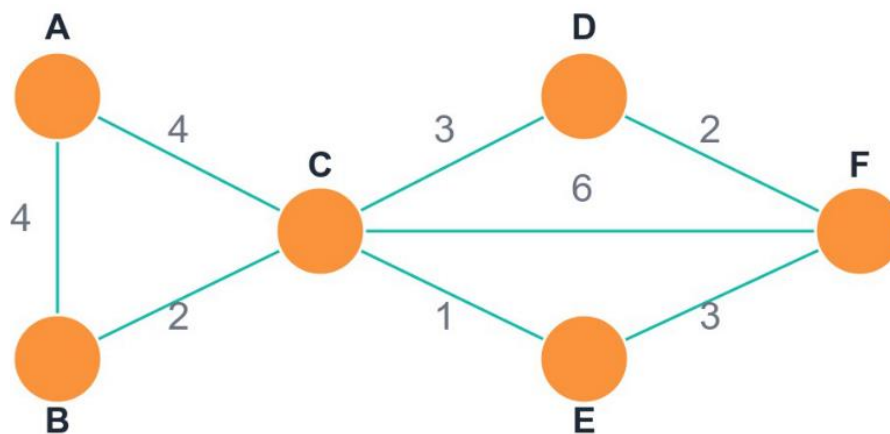
4. In the standard Bellman-Ford algorithm, the outer loop relaxes all edges exactly $V-1$ times. However, for many graphs, the shortest paths are finalized much earlier. Describe a modification or check you can perform during an iteration to terminate the algorithm early without compromising correctness.

5. The Bellman-Ford algorithm runs a relaxation step $V-1$ times.

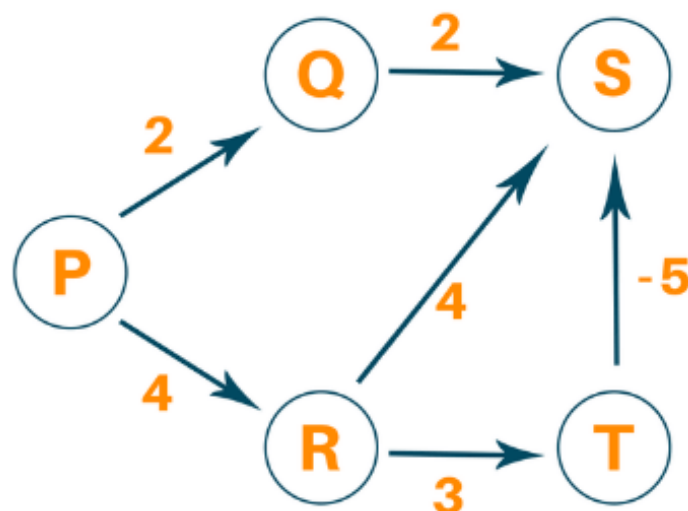
- (a) Explain and show why $V-1$ iterations are sufficient to guarantee the shortest paths are found if no negative-weight cycles exist.
- (b) Then Explain why executing a V -th iteration allows us to identify the presence of a negative-weight cycle reachable from the source.

6. Briefly state the primary difference between the objectives of Warshall's algorithm and Floyd's algorithm.

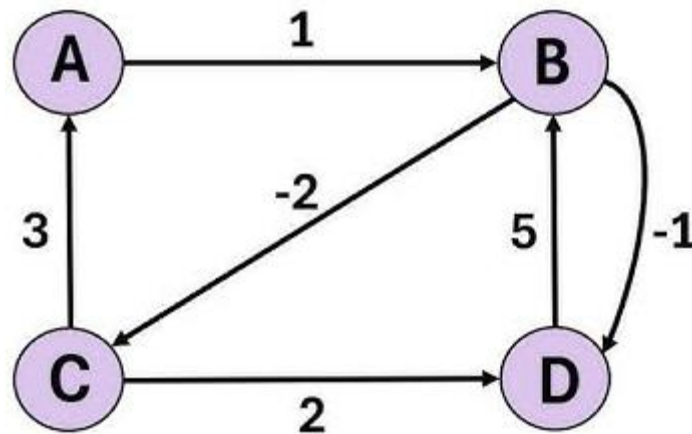
7. Apply Dijkstra's algorithm source F. Find the minimum cost and path from F to A.



8. Apply Bellman-Ford algorithm for the following graph using Q as source.

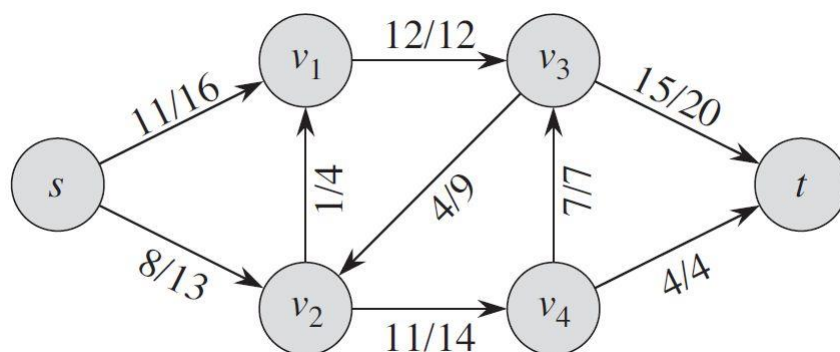


9. Can Dijkstra work with negative edge? Explain.
10. What is the main advantage of Bellman-Ford over Dijkstra's algorithm? How many iterations does Bellman-Ford perform?
11. Can Bellman-Ford work on an undirected graph with a negative edge? Explain.
12. Can Floyd's algorithm handle negative edge weights (without negative cycles)?
13. Which algorithm is best when you need shortest paths between all pairs of vertices in a dense graph?
14. What is the difference between single-source and all-pairs shortest path problems?
15. Determine all pair shortest path from given graph.



Segment 8: Max-Flow, NP-Completeness, Backtracking, Branch and Bound

1. What is flow conservation and capacity constraint?
2. What is a **residual network**? Form a residual network from the following flow graph.



3. How can you increase the flow in a flow network by using augmenting path?

4. Consider a flow network $G = (V, E)$ with source s and sink t . The capacities of the directed edges are given below:

$(s, v1) = 16$
 $(s, v2) = 13$
 $(v1, v2) = 10$
 $(v1, v3) = 12$
 $(v2, v1) = 4$
 $(v2, v4) = 14$
 $(v3, t) = 20$
 $(v4, v3) = 7$
 $(v4, t) = 4$

(a) Execute the Ford-Fulkerson algorithm step-by-step. For each iteration, state the augmenting path selected, its bottleneck capacity, and update the residual graph network.

(b) Identify the final Minimum Cut (S, T) produced by your execution. List the vertices belonging to subset S and subset T , and show mathematically that the capacity of this cut matches your calculated maximum flow.

(c) Suppose the capacity of edge $(v1, v3)$ is increased from 12 to 16. Determine whether the maximum flow value increases. Explain why or why not using structural properties of the network.

5. A backtracking algorithm solves the N-Queens problem by placing one queen in each row and, for every row, trying the columns in increasing order.

(a) Two queens occupy rows i and j in columns c_i and c_j , respectively. Derive the algebraic condition on these indices under which the two queens lie on a common diagonal, and explain why placing exactly one queen per row makes any row-conflict check unnecessary.

(b) Describe the structure of the state-space tree the algorithm explores, and explain precisely what distinguishes a node that is abandoned (pruned) from one that is expanded further.

(c) A classmate argues that backtracking and the brute-force method that lists all N^N column assignments share the same worst-case time complexity, so backtracking offers no real advantage. Assess this argument, distinguishing clearly between worst-case asymptotic cost and observed running time on typical inputs.

6. Consider a 3×3 chessboard.

A backtracking algorithm attempts to place 3 queens so that no two queens attack each other.

(a) Place a queen in row 1, column 1 and continue the search.

(b) Show where backtracking occurs.

(c) Determine whether the 3-Queens problem has a solution and justify your answer.

7. In the 4-Queens problem, queens are placed from row 1 downward, columns are tried left to right, and the queen in row 1 is fixed in column 1.
 - (a) Carry out the backtracking search by hand under this starting choice, recording the sequence of attempted placements and marking every point at which the search backtracks. State the outcome, and if no solution results, explain what causes the failure.
 - (b) Without separately tracing the cases in which the first queen starts in columns 3 and 4, use the board's left-right symmetry to deduce the total number of distinct solutions to the 4-Queens problem. Justify the symmetry argument you use.
 - (c) Explain why changing the order in which columns are tried alters how much of the state-space tree is explored but cannot change the final set of solutions.
8. A backtracking algorithm searches for every subset of $W = \{4, 5, 6, 9\}$ that sums to exactly 15, examining the elements in the order given and branching on include / exclude at each one. Each element may be used at most once.
 - (a) Devise a bounding test, stated in terms of the sum accumulated so far and the total of the elements not yet examined, that detects when a partial subset can be safely abandoned. Argue why your test is sound — that is, why it never discards a node that could still lead to a valid subset.
 - (b) Apply your test while exploring the state-space tree, report every subset that sums to 15, and identify at least two nodes that your test allows you to prune.
 - (c) The elements are listed here in increasing order. Explain how this ordering strengthens the pruning, and what advantage is lost if the elements are examined in some other order.
9. Why is Backtracking generally considered unsuited for optimization problems like the Traveling Salesperson Problem compared to Branch and Bound?
10. Backtracking is applied to the subset-sum problem on a set of n positive integers.
 - (a) A student claims that because backtracking prunes the search tree, its worst-case running time is polynomial in n . Construct a specific family of inputs (parameterised by n) on which the bounding function prunes almost nothing, and use it to refute the claim.
 - (b) For your family of inputs, estimate the number of nodes of the state-space tree that are visited, and state the resulting worst-case time complexity.
 - (c) Reconcile this exponential worst case with the observation that backtracking is, in practice, far faster than examining every subset. What property of real inputs does the pruning exploit?
11. Consider the set $S = \{1, 2, 3\}$. A backtracking algorithm generates all subsets by deciding for each element whether to include it or exclude it.
 - (a) Draw the state-space tree for generating all subsets.
 - (b) List all subsets produced by the algorithm.
 - (c) How many leaf nodes are there in the state-space tree? Explain your answer.

12. A graph G has vertex set $\{1, 2, 3, 4\}$ and edge set $\{(1,2), (1,3), (2,3), (2,4), (3,4)\}$. A backtracking algorithm attempts to colour G with $m = 3$ colours, denoted 1, 2, and 3, considering vertices in the order 1, 2, 3, 4 and colours in the order 1, 2, 3.

(a) Trace the algorithm, showing for each vertex every colour that is tried and the reason any colour is rejected, until the first proper colouring is produced.

(b) Determine the chromatic number of G and prove that no smaller number of colours can suffice, referring to a specific structural feature of the graph.

(c) If the vertices were instead processed in the order 4, 2, 3, 1, would the algorithm still produce a proper 3-colouring? Explain what the vertex ordering can and cannot affect.

13. Define the complexity classes P and NP . What is the fundamental difference between a problem being "solvable in polynomial time" vs. "verifiable in polynomial time"?

14. What does it mean for a problem to be NP -Hard? How does an NP -Hard problem differ from an NP -Complete problem?

15. "Every problem in P is also in NP ." Explain your reasoning.

16. If an algorithm takes exponential time $O(2^n)$ to find a solution, but its solution can be verified in $O(n^2)$ time, does this problem belong to the class P , the class NP , or both? Justify your answer.

17. A graph G has vertex set $\{1, 2, 3, 4, 5\}$ and edge set $\{(1,2), (2,3), (3,4), (4,5), (5,1), (1,3), (2,5)\}$. A backtracking algorithm searches for a Hamiltonian cycle that begins and ends at vertex 1.

(a) State the two conditions a vertex must satisfy before it can be appended to the current partial path, and explain why each condition is necessary.

(b) Using these conditions, find one Hamiltonian cycle of G , and exhibit one partial path starting at vertex 1 that satisfies the conditions at every step yet still reaches a dead end. Explain what forces the backtrack in that case.

(c) Explain why the feasibility checks bring the search well below the $(n - 1)!$ orderings of the vertices, yet cannot guarantee a sub-exponential worst case. Relate your answer to the fact that deciding whether a graph is Hamiltonian is NP -complete.

18. Given the set $A = \{2, 3, 5\}$, use backtracking to find all subsets whose sum is 5.

(a) Show the include/exclude decisions made during the search.

(b) Identify every subset whose sum equals 5.

(c) At what point can the algorithm stop exploring a branch if the current sum becomes greater than 5? Explain why.

19. A traveling salesperson needs to visit 4 cities labeled 1, 2, 3 and 4. The directed cost/distance matrix between the cities is defined as follows (where INF represents a non-existent edge):

$$C = \begin{pmatrix} \infty & 4 & 7 & 3 \\ 5 & \infty & 6 & 2 \\ 2 & 3 & \infty & 5 \\ 6 & 1 & 4 & \infty \end{pmatrix}$$

- (a) Using the classical Row and Column Reduction method, calculate the initial cost lower bound for the root node (Node 0) of your state-space tree. Show the reduced matrix explicitly and state the sum of the reduction constants.
- (b) From the root node, branch out to city 2 (representing the path segment 1 → 2). Construct the updated cost matrix for this child node by setting row 1, column 2, and cell (2,1) to INF. Perform any necessary reductions and calculate the exact lower bound for this path segment node.
- (c) Show how a Best-First Search strategy uses these calculated lower bounds to choose which node to expand next, and identify under what condition a branch can be safely pruned (killed) without losing the optimal tour.